ESSnet Big Data

Specific Grant Agreement No 2 (SGA-2)

https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata
http://www.cros-portal.eu/.....

Framework Partnership Agreement Number **11104.2015.006-2015.720**

Specific Grant Agreement Number **11104.2016.010-2016.756**

Work Package 6

Early estimates of economic indicators

Deliverable 6.9

Report and recommendation about IT infrastructure needed for the storage, analysing, combining data sources and the process of calculating early economic indicators

Version 2018-28-05

Prepared by: Henri Luomaranta (Finland), Marco Puts (Netherlands), Grzegorz Grygiel (Poland), Alessandra Righi (Italy), Pedro Campos (Portugal), Črt Grahonja, Tomaž Špeh (SURS)

ESSnet co-ordinator:

Peter Struijs (CBS, Netherlands)
p.struijs@cbs.nl
telephone : +31 45 570 7441
mobile phone : +31 6 5248 7775

Contents

1 Summary 3

2 IT infrastructure for estimating early economic indicators using traffic loops data at SURS 4

2.1 Explanation of idea and data preparation 4

2.1.1 Traffic sensors data 5

2.1.2 Enterprise data 6

2.2 Explanation of data processing 9

2.2.1 Data preparation 9

2.2.2 Nowcasting using principal component analysis and linear regression 24

2.3 Conclusions 33

2.4 Literature 33

1 Summary

The main goal of the WP6 was to explore how a combination of (early available) multiple big data sources, administrative and existing official statistical data could be used in creating an existing or new early estimates for official statistics.

This document focuses on results of the study related to the IT infrastructure needed for the storage, analysing, combining data sources and the process of calculating early economic indicators. It describes IT infrastructure and applications developed within the pilot carried out at Statistics Slovenija (SURS).

This report gives a deep (IT) insight into how to use workable application based on the PCA method for purposes of nowcasting early economic indicators. One of the main results of the whole work in WP6 is the IT application that is ready to use for every NSI interested in this topic.

The relatively small investments that are required are related to modelling skills (in maintaining and updating the models) and adding a few features in the existing IT systems for storing information on the models, results and source data. The code will be available on the https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/index.php/Main_Page. The code can be used free but no maintenance will be provided.

2 IT infrastructure for estimating early economic indicators using traffic loops data at SURS

2.1 Explanation of idea and data preparation

The idea of using the model in Phython, SAS and in R came from colleagues from Statistics Finland. The idea was implemented by colleagues from the Statistical Office of the Republic of Slovenia.

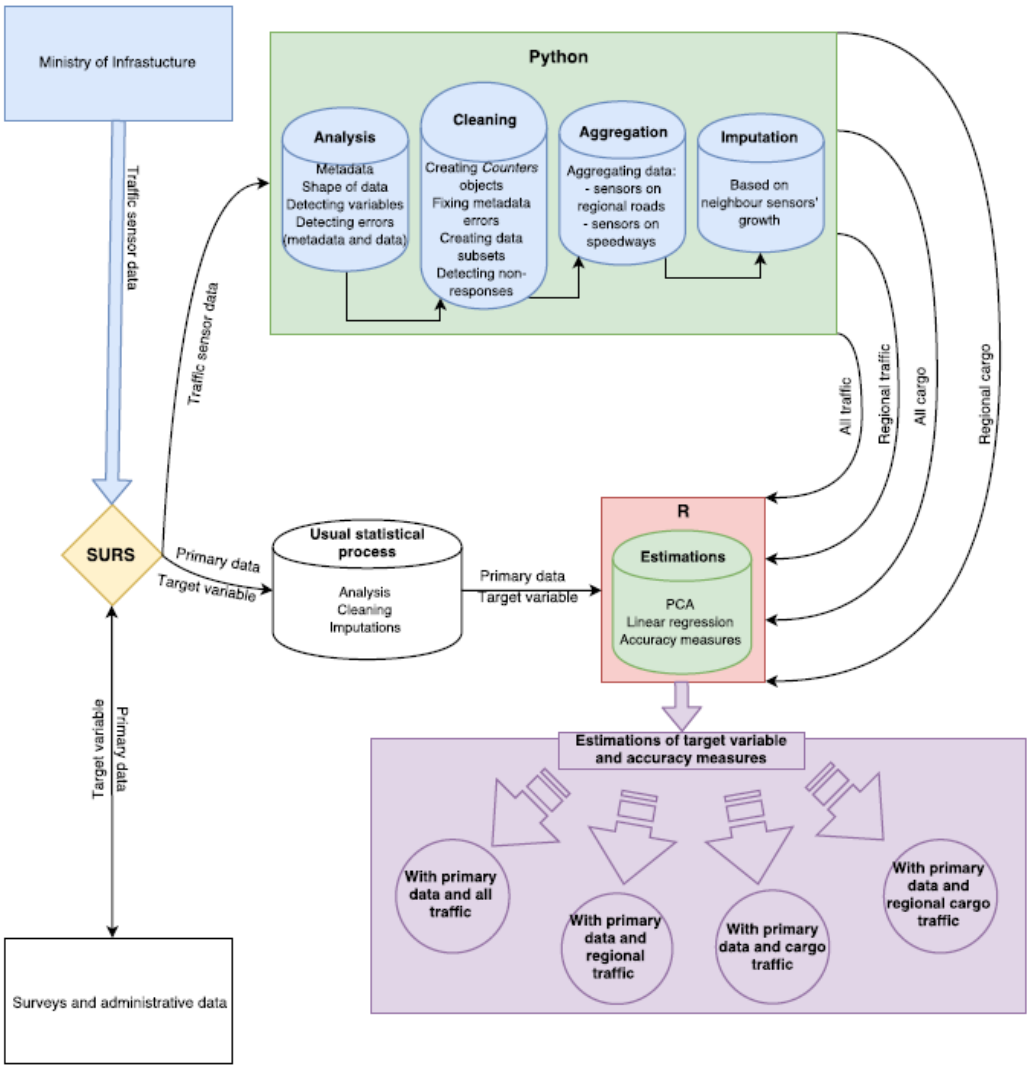
This nowcasting model consists of 3 stages:

1. Principal Component Analysis (PCA) is used to extract principal components from enterprise data. For each enterprise included in the model, time series of data without any missing values is needed. Then, first few principal components are chosen.

2. Linear regression is used: the time series of interest (e.g. GDP) is the dependent variable (Y) and the chosen principal components are the predictors (X1, …, Xn). The seasonal component and other predictors can be added.

3. Traffic sensors data are used as primary and secondary repressor in a linear regression method.

Picture 1: Data processing and nowcasting phases



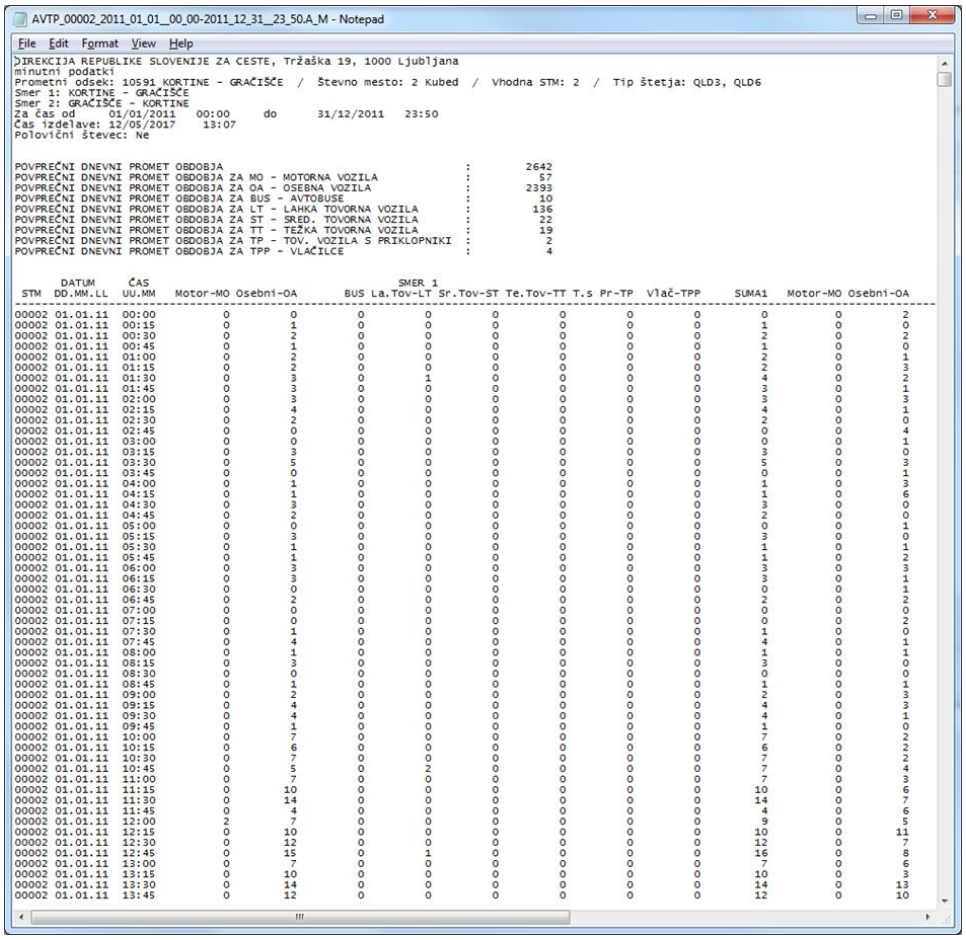
2.1.1 Traffic sensors data

Traffic sensors data are prepared using Python.

That has been already prepared with distributed categories according to the formula that is used at the Ministry of Infrastructure into count spots data. The final product is joint annual data for each count spot, which includes categories for Motorcycles, Personal Vehicles, Light Goods Vehicles, Medium Goods Vehicles, Heavy Goods Vehicles, Semi-Trailers, Tractor-Trailers, and Buses. The vehicle counts are given for 2 directions. However, on speedways and highways some of the count spots only count in one direction (the only available sensors that make up these counting spots are on one side). The available time intervals of vehicle counts are 15-minute, 1-hour or daily intervals.

The 15-minute interval of semi-edited data was chosen for our work. We gained it from the Ministry on an external drive in the form of 3529 text files in 24 GB of size. With the choice of edited data we skipped quite a number of steps towards final data; however, after careful examination it turned out that the text files were not in an easy-to-read format and furthermore there were mistakes in the texts. Nevertheless, we decided to use this source (for now), as the unification of raw sensor data outweighed the work needed to repair mistakes in the files. The chosen time interval of each observation was 15 minutes for the period between 2011 and 2017.

Picture 2: Traffic loops data structure



AVTP_00002_2011_01_01__00_00-2011_12_31__23_50.A_M - Notepad

File Edit Format View Help

```
DIREKCIJA REPUBLIKE SLOVENIJE ZA CESTE, Tržaška 19, 1000 Ljubljana
minutni podatki
Prometni odsek: 10591 KORTINE - GRAČIŠČE  /  Števno mesto: 2 Kubed  /  Vhodna STM: 2  /  Tip štetja: QLD3, QLD6
Smer 1: KORTINE - GRAČIŠČE
Smer 2: GRAČIŠČE - KORTINE
Za čas od      01/01/2011    00:00      do      31/12/2011    23:50
Čas izdelave: 12/05/2017    13:07
Polovični števec: Ne


POVPREČNI DNEVNI PROMET OBDOBJA                                  :        2642
POVPREČNI DNEVNI PROMET OBDOBJA ZA MO - MOTORNA VOZILA           :          57
POVPREČNI DNEVNI PROMET OBDOBJA ZA OA - OSEBNA VOZILA            :        2393
POVPREČNI DNEVNI PROMET OBDOBJA ZA BUS - AVTOBUSE                :          10
POVPREČNI DNEVNI PROMET OBDOBJA ZA LT - LAHKA TOVORNA VOZILA     :         136
POVPREČNI DNEVNI PROMET OBDOBJA ZA ST - SRED. TOVORNA VOZILA     :          22
POVPREČNI DNEVNI PROMET OBDOBJA ZA TT - TEŽKA TOVORNA VOZILA     :          19
POVPREČNI DNEVNI PROMET OBDOBJA ZA TP - TOV. VOZILA S PRIKLOPNIKI :          2
POVPREČNI DNEVNI PROMET OBDOBJA ZA TPP - VLAČILCE                :           4
```

STM	DATUM DD.MM.LL	ČAS UU.MM	SMER 1 Motor-MO	Osebni-OA	BUS	La.Tov-LT	Sr.Tov-ST	Te.Tov-TT	T.s Pr-TP	Vlač-TPP	SUMA1	Motor-MO	Osebni-OA
00002	01.01.11	00:00	0	0	0	0	0	0	0	0	0	0	2
00002	01.01.11	00:15	0	1	0	0	0	0	0	0	1	0	0
00002	01.01.11	00:30	0	2	0	0	0	0	0	0	2	0	2
00002	01.01.11	00:45	0	1	0	0	0	0	0	0	1	0	0
00002	01.01.11	01:00	0	2	0	0	0	0	0	0	2	0	1
00002	01.01.11	01:15	0	2	0	0	0	0	0	0	2	0	3
00002	01.01.11	01:30	0	3	0	1	0	0	0	0	4	0	2
00002	01.01.11	01:45	0	3	0	0	0	0	0	0	3	0	1
00002	01.01.11	02:00	0	3	0	0	0	0	0	0	3	0	3
00002	01.01.11	02:15	0	4	0	0	0	0	0	0	4	0	1
00002	01.01.11	02:30	0	2	0	0	0	0	0	0	2	0	0
00002	01.01.11	02:45	0	0	0	0	0	0	0	0	0	0	4
00002	01.01.11	03:00	0	0	0	0	0	0	0	0	0	0	1
00002	01.01.11	03:15	0	3	0	0	0	0	0	0	3	0	0
00002	01.01.11	03:30	0	5	0	0	0	0	0	0	5	0	3
00002	01.01.11	03:45	0	0	0	0	0	0	0	0	0	0	1
00002	01.01.11	04:00	0	1	0	0	0	0	0	0	1	0	3
00002	01.01.11	04:15	0	1	0	0	0	0	0	0	1	0	6
00002	01.01.11	04:30	0	3	0	0	0	0	0	0	3	0	0
00002	01.01.11	04:45	0	2	0	0	0	0	0	0	2	0	0
00002	01.01.11	05:00	0	0	0	0	0	0	0	0	0	0	1
00002	01.01.11	05:15	0	3	0	0	0	0	0	0	3	0	0
00002	01.01.11	05:30	0	1	0	0	0	0	0	0	1	0	1
00002	01.01.11	05:45	0	1	0	0	0	0	0	0	1	0	2
00002	01.01.11	06:00	0	3	0	0	0	0	0	0	3	0	3
00002	01.01.11	06:15	0	3	0	0	0	0	0	0	3	0	1
00002	01.01.11	06:30	0	0	0	0	0	0	0	0	0	0	1
00002	01.01.11	06:45	0	2	0	0	0	0	0	0	2	0	2
00002	01.01.11	07:00	0	0	0	0	0	0	0	0	0	0	0
00002	01.01.11	07:15	0	0	0	0	0	0	0	0	0	0	2
00002	01.01.11	07:30	0	1	0	0	0	0	0	0	1	0	0
00002	01.01.11	07:45	0	4	0	0	0	0	0	0	4	0	1
00002	01.01.11	08:00	0	1	0	0	0	0	0	0	1	0	1
00002	01.01.11	08:15	0	3	0	0	0	0	0	0	3	0	0
00002	01.01.11	08:30	0	0	0	0	0	0	0	0	0	0	0
00002	01.01.11	08:45	0	1	0	0	0	0	0	0	1	0	1
00002	01.01.11	09:00	0	2	0	0	0	0	0	0	2	0	3
00002	01.01.11	09:15	0	4	0	0	0	0	0	0	4	0	3
00002	01.01.11	09:30	0	4	0	0	0	0	0	0	4	0	1
00002	01.01.11	09:45	0	1	0	0	0	0	0	0	1	0	0
00002	01.01.11	10:00	0	7	0	0	0	0	0	0	7	0	2
00002	01.01.11	10:15	0	6	0	0	0	0	0	0	6	0	2
00002	01.01.11	10:30	0	7	0	0	0	0	0	0	7	0	2
00002	01.01.11	10:45	0	5	0	2	0	0	0	0	7	0	4
00002	01.01.11	11:00	0	7	0	0	0	0	0	0	7	0	3
00002	01.01.11	11:15	0	10	0	0	0	0	0	0	10	0	6
00002	01.01.11	11:30	0	14	0	0	0	0	0	0	14	0	7
00002	01.01.11	11:45	0	4	0	0	0	0	0	0	4	0	6
00002	01.01.11	12:00	2	7	0	0	0	0	0	0	9	0	5
00002	01.01.11	12:15	0	10	0	0	0	0	0	0	10	0	11
00002	01.01.11	12:30	0	12	0	0	0	0	0	0	12	0	7
00002	01.01.11	12:45	0	15	0	1	0	0	0	0	16	0	8
00002	01.01.11	13:00	0	7	0	0	0	0	0	0	7	0	6
00002	01.01.11	13:15	0	10	0	0	0	0	0	0	10	0	3
00002	01.01.11	13:30	0	14	0	0	0	0	0	0	14	0	13
00002	01.01.11	13:45	0	12	0	0	0	0	0	0	12	0	10

2.1.2 Enterprise data

Enterprise data are prepared using SAS.

SAS is used to connect to the server and to take the enterprise data.

There are two possible forms:

- First form: obdobje, P… …

A table with variable obdobje (period of time series) and many variables for separate enterprises is made. Variable obdobje has values e.g. 2008M01 (in case of monthly time series) or 2008Q1 (in case of quarterly time series), and separate variables for enterprises are named e.g. P1000002, where 1000002 means the second company of the first source.

Table 1: Monthly turnover of enterprise data

obdobje	P1000001	P1000002	…	P1000973
2008M01	3526	.	…	66519
2008M02	4252	332	…	36012
…	…	…	…	…
2015M12	5241	412	…	71025

Enterprise data variables can have missing values.

- Second form: obdobje, spr, pris_datum, id, tip_pod

A table with variables obdobje, spr, pris_datum, id, tip_pod is made. Variable obdobje has values e.g. 2008M01 (in case of monthly time series) or 2008Q1 (in case of quarterly time series). Variable spr is the variable of interest (e.g. turnover). Variable pris_datum tells us when the information became available or was received. Variable id has different values for each enterprise (e.g. P1000002, where 1000002 means the second company of the first source). Variable tip_pod can have values s (raw data) or u (edited data).

- If we have data in the second form, they are automatically transformed to the first form later in R.
- If we have monthly enterprise data and quarterly time series of interest data, the enterprise data are automatically transformed from monthly to quarterly using mean[1] later in R.

In case of using Big Data sources (micro level) data must be structured in the same way as enterprises data. Table 2 shows an example where enterprise data (variables P1...) are combined with traffic sensor data (variables P0…).

[1] If e.g. an enterprise variable has value for all the months of the quarter, their mean is the value for the quarter; if a value for at least one month of the quarter is missing, then the value for the quarter is missing.

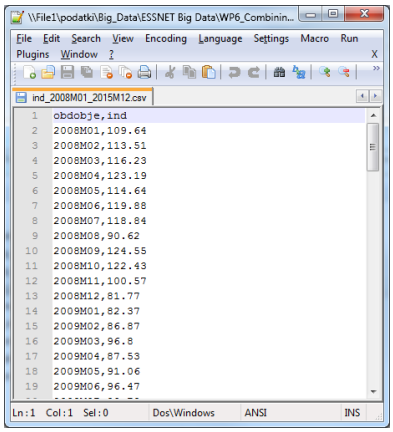
Table 2: Example of combined data from industry survey and traffic loops data

	P1003162	P1003164	P1003166	P1003168	P1003170	P002	P003	P006	P010
1	216218	44818,67	71895,67	56055,67	260288,3	1824,333	7714,333	15132	4092,667
2	245734,7	80276,67	60211	39325,67	217410,7	1419,667	10247,67	19597,67	5346,667
3	296705	47200,67	64541	75624,33	269679	1392	9770,667	19843,33	6352
4	231986,3	55985	94126	65412	276388,7	1088	8770	23579	5138,667
5	264973	38550,67	66172,67	59620,67	169702,7	1098,333	7693,667	19336,67	4771,333
6	276598,3	29820,67	40195,67	76714	182335	1050,667	9514	19132,67	6456,333
7	247292	19871,67	64405,33	88768,67	198847,3	1064,667	9937	17248,33	6919
8	272853	16915	93066,67	78192	229577	883,6667	7929,667	15822	4068,667
9	300986,3	18163,67	52228	51534,33	185970,3	775	6209	12693,33	3409,333
10	272550,7	13266,33	61242,67	47449,67	246491,3	922	8151	16869	4837
11	288410,3	NA	74888	73008,67	284151,3	1226,333	9448,667	19202,67	5885,667
12	290073	NA	69280,33	63277	202978,3	1091,333	7900,667	16063,67	3566,551
13	310173	NA	83583	57082	162391,7	788,3333	4983,333	13022	2827,333
14	316162,7	NA	75582,67	78835,33	147334	1039,667	6120,333	15912,33	5133,889
15	334938,7	NA	61854,67	57812	262239,3	1191,434	5967,333	16233	5925,667
16	375113,7	NA	84696	56782,67	190403,3	922,3333	5605	15378,67	3462,677
17	342045	NA	72032,33	39791,33	115945	736,6667	5063	13228,67	3653,667
18	334273,7	NA	69265	42082,67	136967,7	957,6667	5777,333	16466,67	5296,667
19	386572,7	NA	65479,67	273164,3	215871,3	1241,667	5768,333	16714	6395,333
20	449406	NA	76241	59811,67	129889	986,3333	5590,333	16164,33	3956
21	404387	NA	37974,33	30490,67	NA	753,6667	4553	13784	3841,333
22	444126,7	NA	78102,67	43745,33	NA	1030,667	5870,667	18748,67	5892,667
23	438757,7	NA	86084	39027,67	NA	1148	5895,667	17713,33	7566
24	492560,3	NA	36321	65785	NA	1196,333	5971,667	17673,33	4992,667

2.1.2.1 Time series of interest data

Time series of interest is the time series that we wish to estimate, nowcast. Data are prepared in csv file. It has two variables: variable obdobje has values e.g. 2008M01 (in case of monthly time series) or 2008Q1 (in case of quarterly time series), and variable e.g. ind (the name of the time series of interest) has original values of the time series. The file is stored in the appropriate subfolder.

Example: ind_2008M01_2015M12.csv[2]:



[2] Only time series of interest data can have 1 period less than the other data! E.g. enterprise data and other predictors have data from January 2008 until December 2015, but time series of interest has data from January 2008 until November 2015 or December 2015.

2.1.2.2 Seasonal component and other predictors

Seasonal component can be used as an additional predictor in linear regression. Seasonal component is derived from time series of interest. Other predictors (e.g. traffic loop data at the macro level) can be used as additional predictors in linear regression. They have the same form as time series of interest data.

Example: klima_2008M01_2015M12.csv

2.1.2.3 Model in R

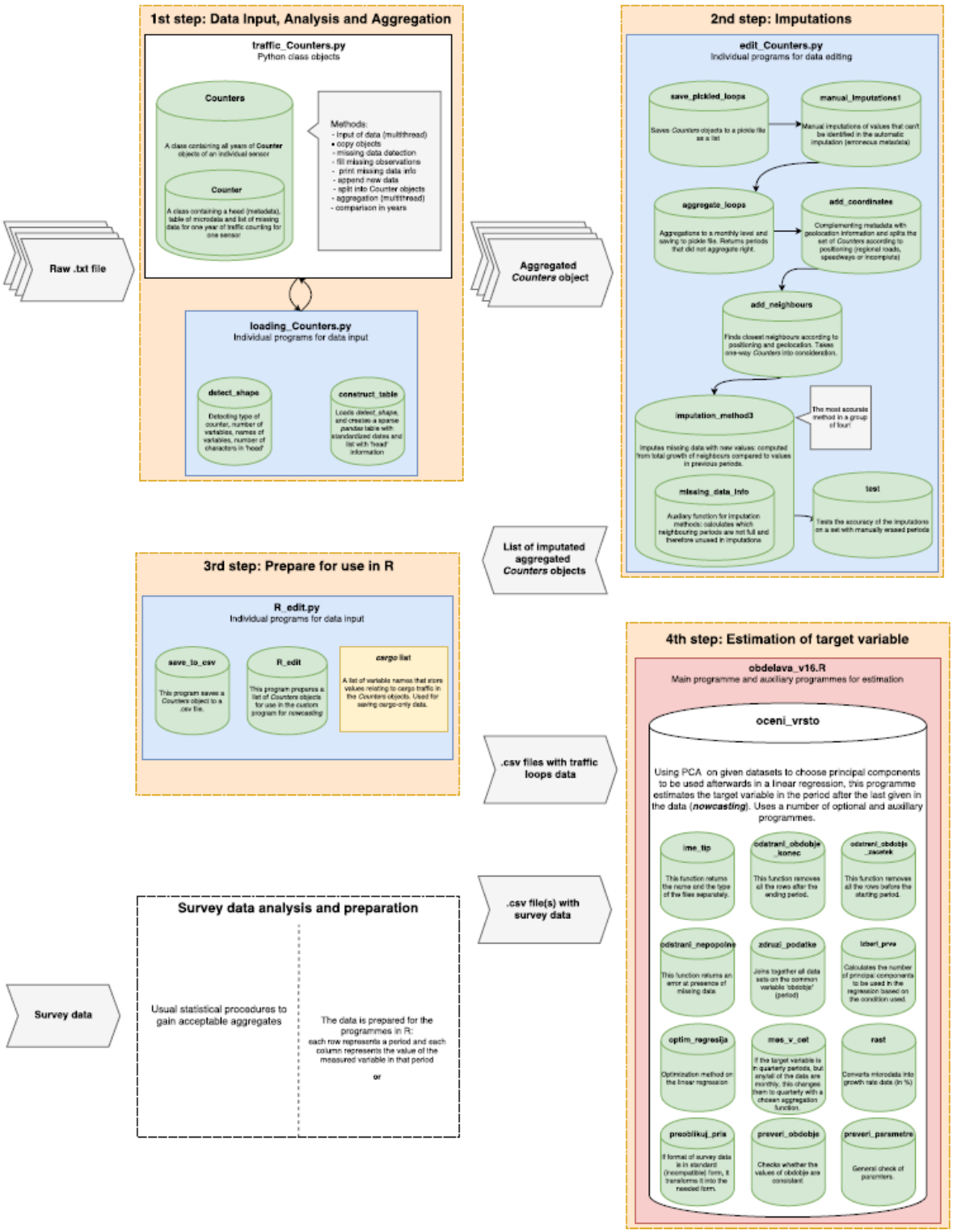
The nowcasting model is programmed in R.

- See obdelava_v15.R in subfolder 3_Obdelava. Use this code to estimate (nowcast) 1 time series of interest many times (or only one time).
- See obdelava_v15_enkrat.R in subfolder 3_Obdelava. Use this code if you want to estimate (nowcast) 1 time series of interest only one time and you want to run separately different parts of the code to see the intermediate results (statistics, charts, etc.).

2.2 Explanation of data processing

2.2.1 Data preparation

Picture 3: Data preparation proces



2.2.1.1 Pyhton module loading_Counters

This document is a documentation file for the Python module **loading_Counters.py**.

The loading_Counters.py module comprises two functions, which read Slovene-style traffic loop data into a structured form, on which other data manipulation techniques can be easily applied. Because of language constraints, these files may only be used on Slovene data; however, we have marked the most obvious language- and region-specific parts of the code to change for use in different countries.

The functions take into consideration different formats of data and use information in the unstructured part of the data text files to determine the right format of the data.

This module represents one of the two core modules, which we use for traffic loop data integration experiments as part of the development regarding exploration of new sources of data (WP6).

This module imports other modules and outside functions to work correctly:

1. module **pandas** (as **pd**)
2. functions **time**, **strptime** (from module **time**)
3. function **day_name** (from module **calendar**)
4. function **datetime** (from module **datetime**)
5. module **warnings**

Programs in the module:

detect_shape: a program that detects the shape of a data file and based on information in the head of the file defines the number of columns and variables in the data as well as the type of data (raw/minute/hourly/daily). Returns error if the number of detected variables is higher than the number of detected column names.

Remark: *if the shape of data is unexpected to the program, it will prompt a manual input with printed detected columns. It is recommended that in the case of looping this program on differently shaped data files you fill the 'shape' parameter accordingly.*

The parameters are (optional parameters in italic):

1. file – the name of the traffic loop text file to read and detect the shape of (type string)
2. path – the directory path to the file (type string)
3. *shape – a manual shape that overrides the program's automatic detection [3]; it has multiple options:*
   - None → the superdomain names are defined as below (*)

[3] ***Caution:*** *changing this may change the behavior of programs in the data manipulation part. The programs use some specific superdomain names: the 'General' superdomain is the group of all non-numeric variables (such as the descriptive variables). Excluding this name will result in the programs trying to sum the values stored in this group of variables!*

- tuple: (number of variables to be grouped, name of group (superdomain)) → manual selection of the names of superdomains. It is advisable that at least one is 'General' (see footnote)

- tuple: (number of variables to be grouped, integer) → semi-manual selection of names; the names will be 'General', 'Channel 1' and *'Channel 2', but the order is set in the same way as above with numbers (e.g. if we want the first 9 values to be grouped under 'General', then we fill [(9, 1),…])*

The outputs of the function are:

1. superdomain – the names of superdomains (variable groups); can be set manually with the *shape* parameter (type list of strings)
2. spot_domain – the detected column names (variables) (type list of strings)
3. start_of_data – number of character before the start of microdata rows (used in the next program)

If parameter *'shape'* is not defined, then the program will try to coerce the columns into groups named *'General', 'Channel 1, 'Channel 2', 'Channel 1 - averages', 'Channel 2 - averages', 'Channel 1 - total average', 'Channel 2 - total average', 'All - total average', 'Speed classes 1'* and *'Speed classes 2'* in case of raw data or *'General', 'Channel 1'* and *'Channel 2'* in case of edited data.

construct_table: a program that constructs a Pandas DataFrame from the text file microdata with variables (domains and superdomains) defined by the **detect_shape** program. If the program does not recognize the time format (HH:MM) or the date format (YYYY/MM/DD, DD/MM/YYYY or DD/MM/YY), then it returns a DataFrame object filled with values but without index and variable names and a description what went wrong. The program returns an *Index Error* if each individual time of observation and/or each variable name are not unique.

The parameters are (optional parameters in italic):

1. file – the name of the traffic loop text file to read and detect the shape of (type string)
2. path – the directory path to the file (type string)
3. *shape – a manual shape that overrides the program's automatic detection defined as above*
4. *day_names – should the DataFrame include weekday names in the index (type Boolean); useful for analysis but increases time in the aggregation process*

The outputs of the function are:

1. the head with metadata (stores the unstructured head from the text file)
2. a sparse Pandas DataFrame object with microdata values indexed with a multi-index: Year, Month, Day, *Day Name(optional)*, Hour, Minutes and column names as detected by the **detect_shape** program

2.2.1.2 Pyhton module traffic_Counters

This document is a documentation file for the Python module file **traffic_Counters.py**.

The **traffic_Counters** module comprises two classes and many methods on each class that represent the data and metadata of traffic loops over some period. The assumption is that the classes are used for Slovene data, which means that some language- and region-specific characteristics need to be changed if the module is to be used in different countries.

The functions take into consideration different formats of data and use information in the unstructured part of the data text files to determine the right format of the data.

This module represents one of the two core modules, which we use for traffic loop data integration experiments as part of the development regarding exploration of new sources of data (WP6).

This module imports other modules and outside functions to work correctly:

1. functions **Process, Queue** (from module **multiprocessing**)
2. function **construct_table** (from module **loading_Counters**)
3. function **time** (from module **time**)
4. function **monthrange** (from module **calendar**)
5. module **pandas** (as **pd**)
6. module **numpy** (as **np**)
7. function **coo_matrix** (from module **scipy.sparse**)
8. module **os**

Objects in the module:

We defined two objects in this class that may be useful for frequent use.

1. **formats_**–known suffixes of text files *(.txt, .csv, .a_m, .s_m, .s_u, .s_d, .ql6*), an elegant way to filter through all of the files in a directory
2. **shape** – the standard shape of Slovene text files with English names

Classes in the module:

Each our file represents one year of data for a specific ***Count spot*** with 15 minute observations. We defined two classes, which represent the core objects in our traffic loops data analysis and editing. The first class, *Counter()*, stores microdata, metadata and missing values information for one year (one file), while the second class, *Counters()*, stores multiple *Counter()* objects to collect data for every year of an individual Count spot.

Counter: is a class of objects that are composed of 3 storage units: the head, the table and the missing_data storage. In the head most of the metadata are stored in this order:

- the year of microdata
- the name of the data file
- the head from the text file including who collected the data, type of data, road section where the data were collected, the directions counted, time of collection, time of construction of the file and whether only one direction was counted
- *if the data are of edited type: the full total and totals for each vehicle type are listed*
- original superdomain and domain (column) names are listed

The microdata are stored as a Pandas DataFrame object in the table. It is recommended that the function **construct_table** from module **loading_Counters** is used for this DataFrame.

The missing_data storage is a dictionary with two keywords: MissingObs and MissingVals. The first one is the collection of all absent observations and the second one is the collection of all unknown values in the table.

Counters: is a class of objects that are a collection of multiple Counter objects with the same code. It can collect different year of **Count spots** or data from multiple text files of the same traffic loop. It has many of the same methods defined as the class Counter, but also additional methods that work on multiple files. Some methods use parallel processing extensively. Since building of a Counters object consists of a collection of Counter objects, it has similar characteristic as a Counter object. They include a head, which is a list of all individual Counter object heads (since features of single Counter data can change, we decided it was better to leave them all in). The tables are the list of all microdata tables from single Counter objects (type Pandas DataFrame). The missing_data storage is a dictionary with MissingObs and MissingVals keywords; however, each of them stores a list of lists for each table present in tables. The lists are ordered in the same way as the tables and furthermore each list has a corresponding number to the table it connects to. In the case of uncollected missing data, the lists are substituted by the non-value None (e.g. the shape of MissingObs and MissingVals is [(0, None), (1, None)…]).

Counters objects can be constructed with a list of Counter objects or by inputting a list of heads and a list of Pandas DataFrame tables of equal length. Specifying missing data is optional; however, if it is done, it needs to follow strict rules:

- A dictionary with MissingObs and MissingVals keywords
- Under each keyword a list of the same length as the number of tables stored
- Elements of these lists need to be tuples with unique elements and also the first element needs to be a positive integer lower than the number of tables in tables

Different methods are available for both Counter objects and Counters objects.

Methods in the class Counter:

head_print: a method for readable printing of Counter object's head. It takes no parameters and it has no other outputs.

MissingObs: return the MissingObs list from the missing_data storage. They take no parameters.

MissingVals: return the MissingVals list from the missing_data storage. They take no parameters.

copy: returns a copy of the current Counter object. Takes additional keyword arguments if a deep copy of the Counter's table is needed.

fill_missing_data: fills the missing_data storage according to non-existence of observations in file and presence of NaN (Not a Number) or None values in microdata. If observation periods are not quarters of an hour, this method raises a ValueError. Warnings are raised if missing data are detected, if unknown data are detected or if the number of observations is higher than expected. Individual variables can be listed to be skipped when checking for missing data. Additionally, a starting and ending period of observations can be specified where the checking is carried out. This is useful for mid-year data editing.

The parameters are (optional parameters in italic):

1. *start – the start of the period to be checked for missing data. This parameter can hold different types of values:*
    - *integer: the number of periods to omit before checking starts*
    - *float: the share of periods to omit from the start of the data*
    - *string starting with 'Q': the starting quarter*
    - *string starting with 'M': the starting month*
    - *tuple or list: the name of the index of the first period to be checked*
2. *end – the end of the period to be checked. This parameter can hold different types of values; see above.* ***Remark:*** *the ending period is included in the check*
3. *dropVar – string or list of strings denominating the variables to be skipped in the missing data check*
4. *other keyword arguments – the arguments for pandas.DataFrame.drop() function that is used in dropVar*

The outputs of the method are:

1. missing_data storage fills with values
2. If the storage has changed, then the method returns **True**

MissingObs_print: prints missing periods in a readable format. Expects a filled missing_data storage and quarter-hour observations! It takes no parameters and it has no other outputs.

from_text: constructs a Counter object from a text file. Optionally missing data are filled into the missing_data storage. This method has the same constraints as the **loading_Counters.construct_table** function.

The parameters are (optional parameters in italic):

4. file – the name of the traffic loop text file to read and detect the shape of (type string)
5. path – the directory path to the file (type string)

6. *shape – a manual shape that overrides the program's automatic detection [4]; it has multiple options:*

   - None → the superdomain names are defined as below (*)

   - tuple: (number of variables to be grouped, name of group (superdomain)) → manual selection of the names of superdomains. It is advisable that at least one is 'General' (see footnote)

   - tuple: (number of variables to be grouped, integer) → semi-manual selection of names; the names will be 'General', 'Channel 1' and *'Channel 2', but the order is set in the same way as above with numbers (e.g. if we want the first 9 values to be grouped under 'General', then we fill [(9, 1),…])*

5. *missing_calc – should missing data be filled-in*
6. *other keyword arguments – the arguments for the* **fill_missing_data** *function*

The outputs of the method are:

1. a new Counter object with data from the text file

fill_with_NaNs: fills the microdata with NaN (Not a Number) or other values where observations are missing. These are read from the missing_data storage.

The parameters are (optional parameters in italic):

1. *nan_value – what value should filled variables be; default is numpy.NaN*

The outputs of the method are:

1. the microdata inflates with missing observations where all variables have the value nan_value

append_observations: method for adding new observations into the Counter's table at the end. Additionally, the missing data check can be run again to update the missing_data storage. An option to warn if new data have different first column values then old data can be set.

The parameters are (optional parameters in italic):

1. newpart – the new microdata (type Pandas DataFrame)
2. *missing_calc - a Boolean to determine whether the* **fill_missing_data** *method should be executed, offers no control over the missing data check*

[4] ***Caution:*** *changing this may change the behavior of programs in the data manipulation part. The programs use some specific superdomain names: the 'General' superdomain is the group of all non-numeric variables (such as the descriptive variables). Excluding this name will result in the programs trying to sum the values stored in this group of variables!*

3. *errors – takes one of two values: 'ignore' or 'raise'. If 'raise' is selected, then the method gives an error if new data are not the same shape as old data or if the first column of new data (usually reserved for codes) has different values than old data*

The outputs of the method are:

1. New data are appended at the end of old data in the Counter's table. The table remains sparse.
2. *If* missing_calc *is selected, then the* missing_data *storage is filled with positions of unknown values and/or missing observations.*

append_from_text: method for adding new observations with the methods **construct_table** and **append_observations** directly from text files.

The parameters are (optional parameters in italic):

5. file – the name of the traffic loop text file to read and detect the shape of (type string)
6. path – the directory path to the file (type string)
7. *shape – a manual shape that overrides the program's automatic detection defined as above*
4. *missing_calc – a Boolean to determine whether the* **fill_missing_data** *method should be executed, offers no control over the missing data check*

The outputs of the method are:

Methods in the class Counters:

copy: returns a copy of the current Counters object. This method takes additional keyword arguments if a deep copy of the Counter's table is needed.

MissingObs: return the MissingObs list from the missing_data storage. They take no parameters.

MissingVals: return the MissingVals list from the missing_data storage. They take no parameters.

split_to_Counters: splits the original Counters object into Counter objects. Does not delete the original Counters object from memory.

No paramters. The output of the method is:

1. a list of Counter objects

add_Counters: integrates other Counter objects into the collection. The heads and tables are expanded and the missing_data storage is updated with the values from **Counter.missing_data**.

The parameters are (optional parameters in italic):

1. counters_list: a list of Counter objects to be added into the Counters object

The output of the method is:

1. The Counters object is updated with new heads, microdata and missing data.

call: calls forward a single Counter object that makes part of the original Counters object or a smaller Counters object with selected Counter objects.

The parameters are (optional parameters in italic):

1. recall: multiple values possible:
    - integer: if smaller than 2000, then returns the Counter that is this number in the order of all Counter objects, if higher than 2000, then the method searches for Counter object with data from the corresponding year
    - string: tries to find the Counter with data from the corresponding year
    - list/tuple of above values: finds Counters with corresponding values according to the above choices
    - range: finds all Counter objects that are in the positions listed in the range

The output of the method is:

1. A Counter or Counters objects depending on the 'recall' parameter.

heads_print: a method to print head/s of the Counters object in a readable manner. Has no other output.

The parameters are (optional parameters in italic):

1. *recall – the recall to a Counter object or a group of Counter objects. Used in the same way as the parameter 'recall' in* **split_to_Counters**.

fill_missing_data: fills missing data from every table in tables. Works in the same way as the **Counter.fill_missing_data** method.

MissingObs_print: prints missing observations in a readable manner. Works the same way as the **Counter.MissingObs_print** method. Can specify Counter object or group of Counter objects to consider.

The parameters are (optional parameters in italic):

1. *recall - the recall to a Counter object or a group of Counter objects. Used in the same way as the parameter 'recall' in* **split_to_Counters**.

fill_with_NaNs: fills the microdata tables with NaN (Not a Number) or other values where observations are missing. These are read from the missing_data storage.

The parameters are (optional parameters in italic):

1. *nan_value – what value should filled variables be; default is numpy.NaN.*

The outputs of the method are:

1. the microdata tables inflates with missing observations where all variables have the value nan_value.

aggregate: aggregates all tables on a time variable. The default aggregating function is summing. Optionally can specify a group of variables to be the only considered in the aggregation. Can additionally aggregate on groups of the time variable (e.g. if aggregating on months, can further specify to aggregate on group of months; these can be repeated, don't need to be in order, etc.). It is possible to drop or keep some variables from the whole set of variables. This method uses parallel processing for faster calculation of aggregates. At the end of the process the function prints the time needed for the operation.

The parameters are (optional parameters in italic):

1. time_level – specify the time variable(s) on the basis of which the data should be aggregated. Can be a single time variable or multiple-level variables. The type of this variable should be the same type as the type of the variable names/values
2. *group – the group that should be aggregated*
3. *sublevel – list of values of the time variable to be grouped together and the basis of aggregation.* ***Remark:*** *the method firstly aggregates on the basis of the time level and then aggregates these aggregated values based on the sublevel groups*
4. *equal_sublevel – if sublevel groups are of equal length, then equal_sublevel takes the number of sublevels to be calculated*
5. *function – the aggregate function, default is 'sum'. String type*
6. *dropVar – variable or list of variables from the bottommost domain to be dropped before aggregating*
7. *keepVar - variable or list of variables from the bottommost domain to be kept before aggregating*
8. *threads – how many should be used in calculating the aggregations*

The outputs of the method are:

1. A Counters object with the same amount of heads and tables as the original object. The values are aggregated on the time level basis.

compare_periods: compare the same given period from at least two of the tables in Counters's tables. Results in an error if less than two tables are selected.

The parameters are (optional parameters in italic):

1. time_level – what time level (variable) should be compared. Can be of the same type as the name of the time variable or a list of such names
2. level_values – which time periods should be compared specifically. The values of this parameter are the name of the time level or a tuple of names for each time level selected. If level_value is not present in the time_level, the method returns a row of NaNs
3. *which – if only a subgroup of Counter objects should be included in the comparision*
4. *function – what aggregation function should be used when comparing higher dimension time levels*
5. *head_text – the head of the resulting Counter object is fully customizable*

The outputs of the method are:

1. a Counter object with as many rows as the factor of selected time levels and Counter objects. Has an empty missing_data storage.

from_texts: construct a Counters objects from texts. Uses parallel processing. Does not fill the missing_data storage. Uses **Counter.from_text**.

The parameters are (optional parameters in italic):

1. directory – the full path to the directory with text files to be read
2. *texts – list specific files to be used for the creation of the Counters object*
3. *start – number of file with which to start the creation of the object (starts with 0)*
4. *end – number of file with which to end the object; the file is included (be mindful that this too starts with 0)*
5. *only_formats – only files with the suffix 'only_formats' will be considered when creating the object.*
   Remark: *parameter 'texts' takes precedence over 'start', 'end' and 'only_formats' parameters.*
6. *shape – see* **Counter.from_text**
7. *threads – number of threads used for parallel processing*

The outputs of the method are:

1. Creates a new Counters object filled with Counter objects with data from the files. The missing_data storage is empty.

Other functions in the module:

pr: print tables in full view. Used when printing tables in PyCharm that can reduce the number of rows and columns shown to save space. The input is the table to be fully printed and has no outputs.

_construct_in_queue: auxiliary function for the **Counter.from_text** method for parallel processing.

The parameters are (optional parameters in italic):

1. k – order number of object going in
2. file – text file from which the data for the **Counter.from_text** is read
3. directory – full path to the directory of the file
4. queue – a queue object for parallel processing
5. shape – the **Counter.from_text** 'shape' parameter

The outputs of the function are:

1. An expression of **Counter.from_text** put in the queue to be executed by one of the threads when parallel processing.

_agg_in_queue: auxiliary function for the **Counters.aggregate** method for parallel processing.

The parameters are (optional parameters in italic):

1. n – order number of object going in
2. Counters_tab – table from tables to aggregate
3. time_level – see **Counters.aggregate**
4. group – see **Counters.aggregate**
5. function – see **Counters.aggregate**
6. dropVar – see **Counters.aggregate**
7. keepVar – see **Counters.aggregate**
8. queue – a queue object for parallel processing

The outputs of the function are:

1. An expression of **Counters.aggregate** put in the queue to be executed by one of the threads when parallel processing.

2.2.1.3 Pyhton module edit_Counters

This document is a documentation file for the Python module file **edit_Counters.py**.

This module contains a lot of language- and region-specific code. Only the most useful programs will be presented here.

This is NOT a core module for traffic loop data integration; however, apart for other purposes, it is also used to impute the data.

This module imports other modules and outside functions to work correctly:

1. module **traffic_Counters** (as **tf**)
2. modules **os**, **pickle**, **gc**, **csv**
3. module **numpy** (as **np**)
4. module **pandas** (as **pd**)
5. function **time** (from module **time**)

Programs in the module:

save_pickled_loops: a program which transforms all text files into Counters objects and then saves them as pickle files. We're in the middle of a data warehouse development, so we had to save files to a hard disk. Since all of the data cannot stay in the computer's memory, we saved the Counters objects in steps to the disk.

The parameters:

1. objects_path – full path to the directory of the text files to be used in the Counters creation
2. save_path – full path to the directory where pickled Counters objects are saved
3. prefix – what common beginning should the files have in the name
4. suffix – list of specific names for pickled objects at the end of the name

The outputs:

1. saves all Counters objects to pickle files in the given directory.

aggregate_loops: unpickle each file and aggregate the tables on a monthly basis. The new, aggregated Counters are saved into a list of pairs (name of Counters object, Counters object) and saved to a pickle file.

add_coordinates: from a file with geo-coordinates the coordinates are saved into each head of all Counters objects. In the process a filter based on position of the Counters data is done, and the list is split into lists of regional road Counters, speedway Counters and Counters that had either more than 1 complete year or 85% of all data missing either was positioned on a junction. These three lists are also the output of the function. It is possible to save the lists to pickles. **Remark:** this function depends on the language used in the Counters's heads.

add_neighbours: for each Counters object from the list finds closest neighbours based on their geometrical position. Vicinity of neighbours is calculated with the Euclidian distance.

The parameters (optional parameters in italic):

1. Counters_list – list of all Counters of a certain type (regional or speedway)
2. *Counters_ – a specific Counters object or list of them for which the neighbours from Counters_list should be found*
3. *num_of_neigh – number of neighbours to be found; default is 4*
4. *save – should the updated list of Counters objects be saved*

missing_data_info: a collection of percents of present data for each month of a Counters object in list for each year of type Pandas DataFrame. Used to find suitable (non-missing valued) neighbours when imputing.

imputation_method1 and **imputation_method3**: imputation methods, described in the document Imputation_methods.docx.

The parameters (optional parameters in italic):

1. Counters_list: list of pairs (name of Counters object, Counters object) for which imputations will be calculated. All neighbours must be in this list (the list should be a closed group)
2. *Counters_: a single Counters object or a sublist of Counter objects (or name, object pair) to be imputed with neighbours from the full Counters_list list. If None, the whole list is taken into account*
3. *imputation_indices – if* **missing_data_info** *has been calculated before, it can be imputed manually as to save time when imputing*

The output:

1. a new list of pairs (name of Counters object, Counters object) with imputed values when needed.

test: a program to test the results of the imputation methods. It erases the percent of present data for each row of a chosen Counters object or each object in a list of Counters objects. It then proceeds with imputations for each row. The output is two Pandas DataFrames with the original and the new imputed values.

The parameters (optional parameters in italic):

1. Counters_list: list of pairs (name of Counters object, Counters object) for which imputations will be calculated. All neighbours must be in this list (the list should be a closed group)
2. method: which method should be tested; can be strings ('1' or '3') or integer (1, 3) or full names in string (e.g 'imputation_method3')
3. *Counters_: a sublist of specific Counter objects to be tested*
4. *imputation_indices – if* **missing_data_info** *has been calculated before, it can be imputed manually as to save time when imputing*

The output:

1. two Pandas DataFrames, one with the original rows of every Counters object's table and one with these rows imputed.

2.2.1.4 Pyhton module R_edit

This document is a documentation file for the Python module file **R_edit.py**.

This is NOT a core module for traffic loop data integration; however, we use the data with linear regression methods in a program of R, where inputs are .csv files. This module has two functions with which we can save ready Counters data into .csv files.

This module imports other modules to work correctly:

1. module **pandas** (as **pd**)
2. modules **pickle**, **time**, **os**

By unpickling Counters objects, the module **edit_Counters** (and by extension the modules **loading_Counters** and **traffic_Counters**) are imported.

Objects in the module:

speed_imp – unpickles the list of speedway Counters. Path is hardcoded.

reg_imp – unpickles the list of regional Counters. Path is hardcoded.

cargo – a list of pairs (superdomain, domain) for all variables which represent cargo vehicles in the tables. We use them to save specific cargo data into .csv files.

Programs in the module:

save_to_csv: save a Counters object into a .csv file with tables appended in the same order as they are stored in tables. Optionally a specific file name, variable names, and date in file name can be selected. It has no other outputs.

The parameters (optional parameters are in italic):

1. Cunters_obj – the Counters object, whose table is to be saved into the .csv file
2. *names – specific names of variables to be used when saving (type list of strings) (default value is hardcoded)*
3. *save_path – location of the .csv file*
4. *name – name of the .csv file*
5. *date – should the current date be appended to the name*
6. *other keyword arguments for the* **pandas.DataFrame.to_csv** *method*

Remark: *currently not working with observation names (time levels) longer than 2 levels.*

R_edit: a program that collects all the tables from Counters objects from the list and creates a single table. A subgroup of variables from each table can be specified and aggregation functions over each table's or over all tables' data can be performed. Returns a Pandas DataFrame with selected data and a list of the names of all variables.

The parameters (optional parameters are in italic):

1. Counters_list – a list of pairs (name of Counters object, Counters object) with Counters objects that are to be collected
2. *subgroup – should the collection happen only on a subgroup of variables*
3. *fun – should the observations of each table be aggregated in some way; if subgroup selects one column of data, nothing happens (type either None or string with aggregation function name)*
4. *total_aggregation – should aggregation be performed over all of the tables and if so with which function (type either False or string with aggregation function name)*
5. *save – should the resulting DataFrame be saved into a .csv file*
6. *keyword arguments for the* **pandas.DataFrame.to_csv** *method*

Outputs of the function:

1. A Pandas DataFrame with selected data from the list of Counters objects
2. The names of the variables (columns) of the DataFrame
3. *If selected, the DataFrame is saved to a .csv file*

2.2.2 Nowcasting using principal component analysis and linear regression

The R code obdelava_v("number of version").R is used to estimate one time series of interest many times. Often, parameters can have different values and the ones that are currently not used are commented (so when one wishes to use them it is just needed to uncomment them).

You can select different beginning (parameter zacetek), ending (parameter v_konec, v_konec_prej), microdata sources (parameter v_pod_vir), time series of interest (parameter vr_vir), condition for choosing principal components (parameter v_izberi_prve_pogoj), optimization direction in linear regression (parameter v_optim_regresija_smer), optimization condition in linear regression (parameter v_optim_regresija_pogoj), seasonal component as a predictor (parameter v_sezona) and other predictors (parameter v_dr_vir), you can take into account the date when information was available or was received (parameter t_plus), etc. The names of output files can have parameter values included or the names can have other suffix (parameter izhod_pripona). All the parameters are set in point 0., then you select the whole code (Ctrl + A) and run it (Ctrl + Enter).

2.2.2.1 Time series of interest

Could be any time series in csv format (with monthly or quarterly data), e.g. Real turnover index in industry:

vr_vir <- "ind_2008M01_2015M12.csv"

This time series must include all the periods from parameter zacetek (beginning of time series) to each element of parameter v_konec (end of time series).

2.2.2.2 Data sources

Micro data sources could be traditional (surveys, etc.) sources or (and) big data sources. In case of big data sources, data must be structured in the same way as data from "traditional" sources.

e.g. Real turnover data for enterprises in industry (data from our sample survey; the data are already edited (imputations, etc.))

v_pod_vir <- c("podatki_vir1.sas7bdat")

Multiple data files can be chosen, e.g.

v_pod_vir <- c("podatki_vir1.sas7bdat","podatki_vir2.sas7bdat","podatki_vir3.sas7bdat")

Data files must still include NAs (because some statistics are computed to compare original data with NAs and data without enterprises with any NAs).

2.2.2.3 Principal Component Analysis

All data that have the value for every period of a testing span (i.e. from the beginning to the end of the testing span) are included as an input in the model. Data time series/variables are standardized (mean = 0, standard deviation = 1). There are different conditions for choosing principal components parameter (parameter v_izberi_prve_pogoj) that will be used as predictors in linear regression later.

Table 3: Conditions for choosing principal components

Condition	Meaning
70	take enough p. c. to explain 70% (or a bit more) of the variability of the enterprise data
75	take enough p. c. to explain 75% (or a bit more) of the variability of the enterprise data
80	take enough p. c. to explain 80% (or a bit more) of the variability of the enterprise data
85	take enough p. c. to explain 85% (or a bit more) of the variability of the enterprise data
90	take enough p. c. to explain 90% (or a bit more) of the variability of the enterprise data
po7	take only as many p. c. to have at least 7 cases (time periods) per independent variable later in the linear regression
po8	take only as many p. c. to have at least 8 cases (time periods) per independent variable later in the linear regression
po10	take only as many p. c. to have at least 10 cases (time periods) per independent variable later in the linear regression
po15	take only as many p. c. to have at least 15 cases (time periods) per independent variable later in the linear regression
po20	take only as many p. c. to have at least 20 cases (time periods) per independent variable later in the linear regression
zadnja5	take every p. c. whose eigenvalue's share among all eigenvalues is greater or

	equal to 5%
Kaiser	take every p. c. whose eigenvalue is greater or equal to 1

2.2.2.4 Linear regression

Y (dependent variable) = time series of interest

X1, …, Xn (predictors) = the chosen principal components and other predictors (parameter v_dr_vir) can also be added. For example, traffic loop data could be used as additional predictor in linear model.

There are other possible optimizations:

- optimization direction (parameter v_optim_regresija_smer)
- optimization condition (parameter v_optim_regresija_pogoj)

2.2.2.5 Different testing spans

Each testing span begins at zacetek. Each testing span has its own ending: one of the elements in v_konec.

e.g. a set of 36 testing spans in case of enterprise data:

- the first period is always 2008M01
- the last period is 2013M01 or 2013M02 or … or 2015M12

Time series of interest might not be available for the last period of the testing span; in this case we cannot estimate the difference between the estimate and the original value. But enterprise data must be available for the whole testing span.

Time series of enterprises without any NAs for a given testing span (i.e. from zacetek to a given element of v_konec) are chosen (balanced method). Their data from zacetek to a given element of v_konec are used to extract principal components. The chosen principal components without the ending (i.e. given element of v_konec) are used for the model. Coefficients from this model and the ending of principal components are used to calculate the estimate of the time series of interest for the ending.

2.2.2.6 Results

The results are presented in 3 output files.

The basic name: OBDELAVA_version_time-series-of-interest_enterprise-data_conditions-for-pca_directions-for-linear-regression_conditions-for-linear-regression_predictor-for-seasonality_test-linear-regression-assumptions_other-predictors (e.g. OBDELAVA_v15_ind_2008M01_2015M12_1_70-75_backward-forward_AIC_stl_NO_klima-ind).

- The first file: basic.csv.

We can see the results for every time span, every condition for choosing principal components, every condition and possible direction for linear regression, and every possibility for predictor for seasonality.

Test for linear regression assumptions is made or not (YES or NO). If we decided for YES, there might be some errors because of this test, so not all the results are saved. If we decided for NO, all the results are saved.

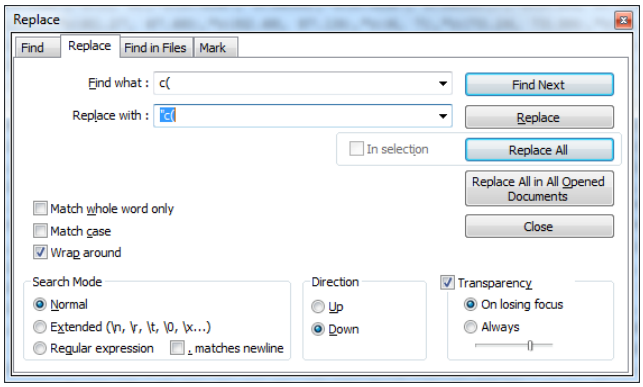
- The second file: basic_skupna.csv.

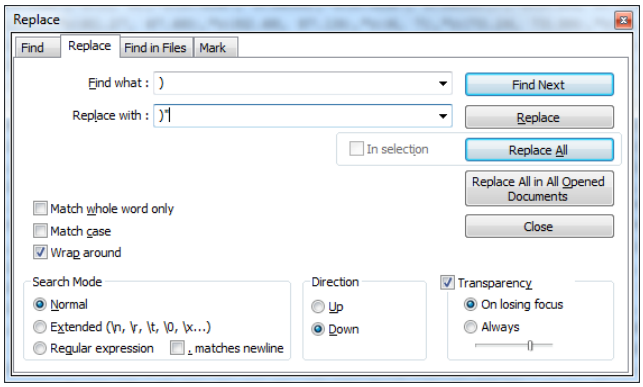
We can see common results for all the results in basic.csv.

- The third file: basic_skupna2.csv.

We can see common results for each set of testing spans with different conditions in basic.csv.

Before opening this file in Excel, It is recommended that one to make a copy (name it e.g. basic_skupna2 - C.csv) and make the following replacements (double quotes have to be around an interval c(number1,number2) so this interval will be in the same cell):





After the replacements, you can open csv in Excel and use Text to columns option with the comma delimiter.

The meaning of variables in the first output file: basic.csv:

Variable(s)	**Meaning**

verzija	Version of code
pot1, pot2, pot3, pot4, pot5	(Parts of) paths
zacetek	The first period in a testing span
konec	The last period in a testing span
konec_prej	The period before the last period in a testing span
v_pod_vir_ref	The names of enterprise data files; the names are separated by hyphen
v_tip_ref	The types of enterprise data; the types are separated by hyphen
podatki_vir_ref	The reference string for enterprise data files
t_plus	Time since the end of the reference period
vr_vir	The time series of interest data file
izberi_prve_pogoj	The condition for choosing principal components
optim_regresija_smer	The direction in function step in optimization of linear regression (backward, forward or both)
optim_regresija_pogoj	The condition for optimization of linear regression (AIC, BIC or NO (i.e. no optimization))
sezona	The option to include seasonal component as predictor (»stl«) or not (»no«).
p_sezona	If seasonal component is included as predictor, this is the p-value of its regression coefficient in regression before optimization.
v_dr_vir_ref	The names of data files of other predictors included in linear regression before optimization; the names are separated by hyphen
drugi_ref	The names of other predictors included in linear regression before optimization; the names are separated by hyphen
p_drugi_s	P-values of other predictors included in linear regression before optimization; p-values are separated by hyphen
p_drugi_d	Share of p-values of other predictors included in linear regression before optimization that are smaller than 0.05 (in %)
per	Periodicity of enterprise data (e.g. 12 for monthly time series)
l_podatki	Length of time series for each enterprise
l_vrsta	Length of time series of interest
st_podj_konec	Number of enterprises that have data for the period konec in the source file
st_podj_podatki	Number of enterprises used for the model (these enterprises have data for the period konec after removing enterprises with any NAs)
perc_st_podj	Share of number of enterprises used for the model (in %)
perc_vsota_podj	Share of value of enterprises used for the model (in %)
n_pca	Number of principal components chosen
perc_pca	How much variability of enterprise data do the chosen principal components explain (in %)
n_pred	Number of predictors after optimization of linear regression
rsqadj1	Adjusted R squared for linear regression before optimization
rsqadj	Adjusted R squared for linear regression (i.e. after optimization)
napake_maxabs	The maximum of absolute values of errors (i.e. minus

	residuals) in linear regression
napake_meanabs	The average of absolute values of errors in linear regression
napake_stod	The standard deviation of errors in linear regression
rel_napake_maxabs	The maximum of absolute values of relative errors (i.e. minus residuals) in linear regression (in %)
rel_napake_meanabs	The average of absolute values of relative errors in linear regression (in %)
rel_napake_stod	The standard deviation of relative errors in linear regression (in %)
vrednost	The original value of time series of interest for the last period of testing span
napoved	The estimate for the original value of time series of interest for the last period of testing span
napaka	The error of estimate for the original value of time series of interest for the last period of testing span (napaka = napoved – vrednost)
napaka_abs	The absolute value of error of estimate for the original value of time series of interest for the last period of testing span
rel_napaka	The relative error of estimate for the original value of time series of interest for the last period of testing span (in %)
rel_napaka_abs	The absolute value of relative error of estimate for the original value of time series of interest for the last period of testing span (in %)
vrednost_rast_1	Growth of original value of time series of interest for the last period of testing span to the original value of the previous period (in %)
vrednost_rast_per	Growth of original value of time series of interest for the last period of testing span to the original value of the same period of the previous year (in %)
napoved_rast_1	Growth of estimate of time series of interest for the last period of testing span to the original value of the previous period (in %)
napoved_rast_per	Growth of estimate of time series of interest for the last period of testing span to the original value of the same period of the previous year (in %)
rast_1_absraz	Absolute value of difference between vrednost_rast_1 and napoved_rast_1 (in percentage changes)
rast_per_absraz	Absolute value of difference between vrednost_rast_per and napoved_rast_per (in percentage changes)
test_predp2	Global Stat of global test of model assumptions (function glvma in package glvma)
p_dw	p-value of Durbin-Watson normality test (function dwtest in package lmtest) (if p-value < 0.05, we can reject independence of residuals in linear regression)
p_dw_1	p-value of Durbin-Watson normality test (function durbinWatsonTest in package car) at lag 1 (if p-value < 0.05, we can reject independence of residuals in linear regression at lag 1)
p_dw_per	p-value of Durbin-Watson normality test (function durbinWatsonTest in package car) at lag per (if p-value < 0.05, we can reject independence of residuals in linear regression

	at lag per)
p_shapiro	p-value of Shapiro-Wilks normality test (function shapiro.test in package stats) (if p-value < 0.05, we can conclude that the residuals in linear regression deviate from normality)
p_ks	p-value of Kolmogorov-Smirnov normality test (function ks.test in package stats) (if p-value < 0.05, we can conclude that the residuals in linear regression deviate from normality)

The meaning of variables in the second output file: basic_skupna.csv:

Variable	**Meaning** (all variables whose name starts with diag are rounded to 2 or 4 digits)
verzija	Version of code
pot1, pot2, pot3, pot4, pot5	(Parts of) paths
zacetek	The first period in all testing spans
zadnji_konec	The last period of the last testing span
st_koncev	Number of testing spans
v_pod_vir_ref	The names of enterprise data files; the names are separated by hyphen
v_tip_ref	The types of enterprise data; the types are separated by hyphen
podatki_vir_ref	The reference string for enterprise data files
t_plus	Time since the end of the reference period
vr_vir	The time series of interest data file
izberi_prve_pogoj	The conditions for choosing principal components; the names are separated by hyphen
optim_regresija_smer	The direction in function step in optimization of linear regression (backward, forward, both, or /); the names are separated by hyphen
optim_regresija_pogoj	The conditions for optimization of linear regression (AIC, BIC or NO (i.e. no optimization)); the names are separated by hyphen
sezona	The options to include seasonal component as predictor (»stl«) or not (»no«); the names are separated by hyphen
v_dr_vir_ref	The names of data files of other predictors included in linear regression before optimization; the names are separated by hyphen
drugi_ref	The names of other predictors included in linear regression before optimization; the names are separated by hyphen
per	Periodicity of enterprise data (e.g. 12 for monthly time series)
diag_st_podj_podatki_range	Minimum and maximum (considering all simulations) number of enterprises used for the model
diag_perc_st_podj_range	Minimum and maximum (considering all simulations) share of number of enterprises used for the model (in %)
diag_perc_vsota_podj_range	Minimum and maximum (considering all simulations) share of value of enterprises used for the model (in %)
diag_n_pca_range	Minimum and maximum (considering all simulations) number of chosen principal components

diag_perc_pca_range	Minimum and maximum (considering all simulations) variability of enterprise data explained by the chosen principal components (in %)
diag_n_pred_range	Minimum and maximum (considering all simulations) number of predictors after optimization of linear regression
diag_rsqadj_range	Minimum and maximum (considering all simulations) adjusted R squared for linear regression (i.e. after optimization)
diag_napake_maxabs	The maximum (considering all simulations) absolute value of errors (i.e. minus residuals) in linear regression
diag_rel_napake_maxabs	The maximum (considering all simulations) absolute value of relative errors (i.e. minus residuals) in linear regression (in %)
diag_napake_meanabs	The average (considering all simulations) of average of absolute values of errors in linear regression
diag_rel_napake_meanabs	The average (considering all simulations) of average of absolute values of relative errors in linear regression (in %)
diag_napaka_maxabs	The maximum (considering all simulations) of absolute values of errors of estimate for the original value of time series of interest for the last period of testing span
diag_rel_napaka_maxabs	The maximum (considering all simulations) of absolute values of relative errors of estimate for the original value of time series of interest for the last period of testing span (in %)
diag_napaka_meanabs	The average (considering all simulations) of absolute values of the errors of estimate for the original value of time series of interest for the last period of testing span
diag_napaka_meansq	The average (considering all simulations) of squared values of the errors of estimate for the original value of time series of interest for the last period of testing span
diag_rel_napaka_meanabs	The average (considering all simulations) of absolute values of relative errors of estimate for the original value of time series of interest for the last period of testing span (in %)
diag_rel_napaka_meansq	The average (considering all simulations) of squared values of relative errors of estimate for the original value of time series of interest for the last period of testing span (in squared %)
diag_d_napake_maxabs	Share of time spans (considering all simulations) for which absolute value of error of estimate for the original value of time series of interest for the last period in smaller or equal to maximum absolute value of errors in linear regression; and 1 minus this share
diag_d_napake_meanabs	Share of time spans (considering all simulations) for which absolute value of error of estimate for the original value of time series of interest for the last period in smaller or equal to average of absolute values of errors in linear regression; and 1 minus this share
diag_d_napake_stod	Share of time spans (considering all simulations) for which absolute value of error of estimate for the original value of time series of interest for the last period in smaller or equal to the standard deviation of errors in linear regression; and 1 minus this share
diag_rast_1_absraz_range	Minimum and maximum (considering all simulations) absolute value of difference between vrednost_rast_1 and

	napoved_rast_1 (in percentage changes)
diag_rast_1_absraz_mean	The average (considering all simulations) of absolute value of difference between vrednost_rast_1 and napoved_rast_1 (in percentage changes)
diag_rast_1_absraz_meansq	The average (considering all simmulations) of squared value of difference between vrednost_rast_1 and napoved_rast_1 (in squared percentage changes)
diag_rast_per_absraz_range	Minimum and maximum (considering all simulations) absolute value of difference between vrednost_rast_per and napoved_rast_per (in percentage changes)
diag_rast_per_absraz_mean	The average (considering all simulations) of absolute value of difference between vrednost_rast_per and napoved_rast_per (in percentage changes)
diag_rast_per_absraz_meansq	The average (considering all simulations) of squared value of difference between vrednost_rast_per and napoved_rast_per (in squared percentage changes)
diag_d_test_predp2	Share of time spans (considering all simulations) for which Global Stat of global test of model assumptions (function glvma in package glvma) is equal to »Assumptions acceptable.«; and 1 minus this share
diag_d_p_dw	Share of time spans (considering all simulations) for which p-value of Durbin-Watson normality test (function dwtest in package lmtest) is greater or equal to 0.05; and 1 minus this share
diag_d_p_dw_1	Share of time spans (considering all simulations) for which p-value of Durbin-Watson normality test (function durbinWatsonTest in package car) at lag 1 is greater or equal to 0.05; and 1 minus this share
diag_d_p_dw_per	Share of time spans (considering all simulations) for which p-value of Durbin-Watson normality test (function durbinWatsonTest in package car) at lag per is greater or equal to 0.05; and 1 minus this share
diag_d_p_shapiro	Share of time spans (considering all simulations) for which p-value of Shapiro-Wilks normality test (function shapiro.test in package stats) is greater or equal to 0.05; and 1 minus this share
diag_d_p_ks	Share of time spans (considering all simulations) for which p-value of Kolmogorov-Smirnov normality test (function ks.test in package stats) is greater or equal to 0.05; and 1 minus this share
diag_d_p_sezona	Share of time spans (considering all simulations) for which p-value of seasonal component's predictor's coefficient is smaller than 0.05; and 1 minus this share
diag_p_drugi_d_mean	Average of shares of p-values (considering all simulations) of other predictors included in linear regression before optimization that are smaller than 0.05 (in %)

The meaning of variables in the third output file (basic_skupna2.csv) is the same as in the second (basic_skupna.csv), but it is calculated for smaller groups (i.e. all time spans that have the same conditions and parameters, etc.).

2.3 Conclusions

This report gives a deep (IT) insight into how to use workable application based on the PCA method for purposes of nowcasting early economic indicators. Although methods for nowcasting early statistics have been known for a long time, we can consider them as "big data methods" due to the fact that we started to employ them when we faced big data (time series). Although there is a range of methods that could be used for the purposes of nowcasting, it is not easy to prepare the IT solution in such a way that it could be used by other statistical institutes. One of the main results of the whole work in WP6 is the IT application that is ready to use for every NSI interested in this topic. The relatively small investments that are required are related to modelling skills (in maintaining and updating the models) and adding a few features in the existing IT systems for storing information on the models, results and source data. However, it has not to be forgotten that the time series of big data is needed in order to employ this kind of sources in nowcasting models.

2.4 Literature

1. pandas: powerful Python data analysis toolkit; https://pandas.pydata.org/pandas-docs/stable/index.html,
2. multiprocessing – Process-based "threading" interface; https://docs.python.org/2/library/multiprocessing.html
3. New steps in big data for traffic and transport statistics, CBS; https://www.cbs.nl/en-gb/our-services/innovation/nieuwsberichten/recente-berichten/new-steps-in-big-data-for-traffic-and-transport-statistics ,
4. Traffic Loop Data, M. Puts, B. Meindl, L. Ma, P. Del Rey; https://statswiki.unece.org/download/attachments/109252755/Marco%20Puts.pdf?version=1&modificationDate=1425992553561&api=v2,
5. Traffic loop data for transport statistics, P. Struijs; https://unstats.un.org/unsd/trade/events/2014/Beijing/presentations/day1/afternoon/4.%20Statistics%20Netherlands%20Traffic%20Loop%20Data%20for%20Transport%20St.pdf